

Análisis sobre una estrategia para Hex

1. Introducción

El juego de Hex presenta un desafío fascinante en el dominio de la Inteligencia Artificial. A diferencia de los juegos combinatorios estándar donde las evaluaciones localizadas del tablero son directas, Hex se define por su conectividad global. Una sola pieza puede alterar drásticamente la evaluación del tablero, uniendo componentes dispares y cambiando el equilibrio de poder. Este informe analiza una arquitectura de IA específica diseñada para jugar a Hex, centrándose en el algoritmo de Búsqueda en Árbol de Monte Carlo (MCTS, por sus siglas en inglés), sus fundamentos teóricos y optimizaciones estructurales.

2. La Justificación de MCTS sobre los Enfoques Tradicionales

En la IA clásica de juegos, algoritmos como Minimax (a menudo mejorados con la poda Alfa-Beta) han sido históricamente el estándar de oro. Sin embargo, aplicar Minimax a Hex revela dos limitaciones severas, a menudo insuperables:

2.1 La Explosión del Espacio de Estados

Minimax se basa en recorrer el árbol de juego hasta una cierta profundidad. En un juego de Hex de $N \times N$, el factor de ramificación inicial es N^2 . Para un tablero estándar de 11x11, el primer jugador tiene 121 movimientos posibles, el segundo tiene 120, y así sucesivamente. El número de posibles estados del tablero crece de manera factorial, lo que hace que la búsqueda exhaustiva sea

computacionalmente intratable más allá de los primeros movimientos. Incluso con la poda Alfa-Beta, el factor de ramificación sigue siendo demasiado grande para tableros de tamaño arbitrario.

2.2 El Cuello de Botella Heurístico

Debido a que Minimax no puede alcanzar estados terminales (el final del juego) en un tiempo razonable, requiere una función de evaluación estática (una heurística) para puntuar los estados no terminales del tablero. Diseñar una heurística precisa para Hex es excepcionalmente difícil. Las métricas tradicionales, como el conteo de piezas o los patrones locales, fallan porque Hex es puramente un juego de conectividad global. Formular una heurística que evalúe con precisión las conexiones virtuales y las ventajas topológicas es matemática y lógicamente complejo.

2.3 La Ventaja de MCTS

MCTS elude ambos problemas. En lugar de explorar todas las ramas por igual hasta una profundidad fija, hace crecer el árbol de búsqueda de forma asimétrica, explorando más profundamente las líneas de juego más prometedoras. Además, elimina por completo la necesidad de funciones de evaluación complejas y específicas del dominio. Al simular partidas de forma aleatoria hasta su conclusión, MCTS confía en la probabilidad estadística de victoria derivada de los estados terminales, reemplazando las heurísticas diseñadas por humanos con muestreo estocástico.

3. Fundamentos Teóricos de MCTS

Desde la perspectiva de la teoría de las Ciencias de la Computación, MCTS es un algoritmo de búsqueda de “el mejor primero” (best-first search) que utiliza simulaciones aleatorias para aproximar el valor teórico de los movimientos en el juego. El algoritmo opera continuamente en un bucle que contiene cuatro fases distintas:

1. **Selección:** El algoritmo recorre el árbol de búsqueda desde la raíz hasta un nodo hoja seleccionando los nodos hijos más prometedores.
2. **Expansión:** Si el nodo hoja seleccionado no es un estado terminal, se añade un nuevo nodo hijo al árbol que representa un movimiento legal no intentado.
3. **Simulación (Playout):** A partir del nodo recién expandido, el algoritmo simula el resto de la partida. En su forma más pura, los movimientos se seleccionan uniformemente al azar hasta que se cumple una condición terminal (victoria/derrota).
4. **Retropropagación (Backpropagation):** El resultado de la simulación se propaga hacia atrás a lo largo del camino hasta el nodo raíz. Cada nodo en el camino actualiza sus datos estadísticos (visitas totales y victorias totales).

3.1 La Fórmula UCB1 (Límite Superior de Confianza)

El motor matemático central de la fase de Selección es el algoritmo UCB1, derivado del problema del bandido multiarmado (Multi-Armed Bandit). Este resuelve de manera elegante el dilema de *exploración versus explotación*. La fórmula se define como:

$$UCB1 = \frac{w_i}{n_i} + c \sqrt{\frac{\ln(N_i)}{n_i}}$$

Donde:

- w_i = El número de victorias simuladas a través del nodo i .

- n_i = El número de veces que el nodo i ha sido visitado.
- N_i = El número de veces que el nodo padre ha sido visitado.
- c = La constante de exploración (a menudo ajustada a valores derivados teóricamente como $\sqrt{2}$).

Interpretación Matemática:

- **El Término Izquierdo ($\frac{w_i}{n_i}$):** Este es el término de *explotación*. Representa la tasa histórica de victorias del nodo. De forma natural, sesga al algoritmo para seleccionar movimientos que han demostrado tener éxito en simulaciones previas.
- **El Término Derecho ($c\sqrt{\frac{\ln(N_i)}{n_i}}$):** Este es el término de *exploración*. Debido a que n_i (visitas al hijo) está en el denominador, este valor se vuelve mayor *cuanto menos* se visita un nodo en relación con su padre. Garantiza matemáticamente que ningún movimiento no intentado o raramente intentado sea ignorado para siempre, asegurando la convergencia asintótica hacia el movimiento óptimo dado un tiempo infinito.

3.2 Por Qué los Científicos de la Computación Consideran Superior a MCTS (Métricas Teóricas)

Más allá de su mecánica básica, MCTS —específicamente cuando UCB1 se aplica a árboles (un algoritmo conocido formalmente como UCT)— está muy bien valorado en Ciencias de la Computación debido a sus estrictas garantías matemáticas y métricas de rendimiento medibles que definen su eficiencia:

- **Límites de Minimización del Arrepentimiento (Regret):** En la teoría algorítmica de juegos, el “arrepentimiento” es la diferencia matemática entre la recompensa obtenida por el algoritmo y la recompensa que se podría haber obtenido si siempre se hubiera jugado el movimiento teóricamente perfecto. Los científicos de la computación favorecen a UCB1 porque garantiza

matemáticamente un **límite superior logarítmico sobre el arrepentimiento**. Esto significa que a medida que el algoritmo se ejecuta, la tasa a la que explora movimientos “malos” disminuye logarítmicamente. Es el límite inferior teórico para esta clase de problemas, lo que significa que ningún algoritmo puede equilibrar la exploración y la explotación de manera más eficiente.

- **Convergencia Asintótica al Óptimo de Minimax:** Una métrica vital para cualquier algoritmo de búsqueda es su propiedad de convergencia. Se ha demostrado matemáticamente (Kocsis y Szepesvári, 2006) que a medida que el número de iteraciones se acerca al infinito ($N \rightarrow \infty$), la probabilidad de que MCTS seleccione el verdadero movimiento óptimo en teoría de juegos converge exactamente a 1. Está garantizado que MCTS encontrará la estrategia perfecta de Minimax sin necesidad de construir nunca un árbol Minimax completo.
- **Reducción del Factor de Ramificación Efectivo (b^*):** Algoritmos tradicionales como la Búsqueda en Anchura (Breadth-First Search) hacen crecer el árbol de búsqueda simétricamente, evaluando $O(b^d)$ nodos (donde b es el factor de ramificación y d es la profundidad). MCTS hace crecer el árbol *asimétricamente*, centrando simulaciones computacionalmente pesadas en ramas matemáticamente prometedoras. En el análisis teórico, esto reduce drásticamente el **Factor de Ramificación Efectivo (b^*)**. Incluso si el factor de ramificación bruto de Hex es superior a 100, MCTS se comporta matemáticamente como si el factor de ramificación fuera exponencialmente menor, lo que le permite buscar líneas altamente relevantes con docenas de movimientos de profundidad.
- **Evaluación Estocástica Independiente del Dominio:** MCTS reemplaza una función heurística estática $h(n)$ por un valor esperado probabilístico $v(n)$, calculado como $\lim_{n \rightarrow \infty} \frac{w_i}{n_i} = v(n)$. Esta métrica demuestra que el algoritmo es un “solucionador generalizado”. Se basa en la Ley de los Grandes Números en lugar del conocimiento del dominio programado por humanos, convirtiéndolo en una IA teóricamente pura capaz de adaptarse a juegos con estados intermedios matemáticamente opacos.

4. Optimizaciones Algorítmicas y Características de Rendimiento

Para cerrar la brecha entre la corrección teórica y el rendimiento práctico, la implementación emplea varias estructuras de datos avanzadas y paradigmas arquitectónicos.

4.1 Unión de Conjuntos Disjuntos (DSU) para la Conectividad

En Hex, comprobar si hay un ganador requiere verificar si un camino contiguo de piezas conecta dos bordes opuestos. Un enfoque ingenuo ejecutaría una Búsqueda en Profundidad (DFS) o una Búsqueda en Anchura (BFS) en cada paso de cada simulación, tomando un tiempo $O(V + E)$. Dadas millones de simulaciones, esto es un inmenso cuello de botella.

Esta implementación reemplaza el recorrido del grafo con una estructura de datos de Unión de Conjuntos Disjuntos (DSU, por sus siglas en inglés).

- **Bordes Virtuales:** Los bordes del tablero se representan como “nodos virtuales”. Un jugador gana si sus dos nodos virtuales respectivos pertenecen al mismo conjunto.
- **Validación en Tiempo Casi Constante:** Cuando se coloca una pieza, se fusiona con las piezas aliadas adyacentes utilizando unión por rango (union-by-rank) y compresión de caminos (path compression). Esto permite que las comprobaciones de la condición de victoria operen en tiempo $O(\alpha(n))$, donde α es la función inversa de Ackermann, lo que en la práctica equivale a $O(1)$ para cualquier tamaño observable del universo.

4.2 Inicialización Perezosa (Lazy Initialization)

El DSU se implementa con inicialización perezosa (lazy initialization) utilizando tablas hash (hash maps) en lugar de matrices preasignadas. En un tablero de $N \times N$, especialmente durante las simulaciones tempranas del juego, muchas celdas permanecen

vacías. Asignar memoria y rastrear el estado de las N^2 celdas por adelantado incurre en una alta sobrecarga. La inicialización perezosa asegura que los ciclos de CPU y la memoria solo se consuman para los elementos que realmente participan en el juego, lo que produce mejoras de rendimiento significativas en tableros grandes.

4.3 Arquitectura de Algoritmo “Anytime” (En Cualquier Momento)

El algoritmo está diseñado como un algoritmo “Anytime” (que puede interrumpirse y dar una respuesta válida). En lugar de definir un número fijo de iteraciones de búsqueda, la implementación utiliza una arquitectura multiproceso (multithreaded).

- Un hilo de trabajo en segundo plano (background worker thread) ejecuta el bucle MCTS de forma continua.
- El hilo principal actúa como un estricto temporizador de hardware.
- Una vez que expira el límite de tiempo exacto, el hilo principal detiene al trabajador.

Este enfoque garantiza que la IA siempre devolverá un movimiento válido dentro de límites de tiempo estrictos (crucial para entornos competitivos) mientras maximiza el número de iteraciones MCTS posibles en el hardware específico que ejecuta el código.
